



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

MODELAGEM DE SISTEMAS

AULA 02

Introdução à modelagem de sistemas

KELIVEN DEMARCHI

keliven.demarchi@edu.sc.senai.br

professorakeliven@gmail.com



PARTE I

Modelagem de Sistemas

Modelagem de sistemas refere-se à criação de representações visuais ou abstratas que ajudam na compreensão, comunicação e análise de sistemas complexos.

Por que é importante?

A modelagem é essencial para projetar, analisar requisitos, comunicar ideias e garantir o alinhamento entre desenvolvedores e stakeholders.

Conceitos Fundamentais

Definição de Modelagem de Sistemas:

Linguagem gráfica para visualização, especificação, construção e documentação de artefatos de sistemas complexos de software.

Proporciona uma forma-padrão para a preparação de planos de arquitetura de projetos de sistemas, incluindo aspectos conceituais tais como processos de negócios e funções do sistema, além de itens concretos como as classes escritas em determinada linguagem de programação, esquemas de bancos de dados e componentes de software reutilizáveis.

Conceitos Fundamentais

Definição de Modelagem de Sistemas:

Atualmente, a modelagem de sistemas se tornou a representação de um sistema usando algum tipo de notação gráfica, que hoje em dia quase sempre são baseadas em notações em Unified Modeling Language (UML).

A modelagem de sistemas ajuda o analista a entender a funcionalidade do sistema e os modelos são usados para comunicação com os clientes.

Conceitos Fundamentais

Objetivo de Modelagem de Sistemas:

- ✓ Aprender o que é UML e por que a UML é relevante para o processo de desenvolvimento de sistemas de software complexos.
- ✓ Dominar o vocabulário, as regras e as expressões da UML, além de aprender, de uma forma geral, como "falar" a linguagem de maneira efetiva.
- ✓ Compreender como aplicar a UML para a solução de vários problemas básicos de modelagem.

Conceitos Fundamentais

Propósito da Modelagem de Sistemas:

O principal propósito é fornecer uma visão visual clara e compreensível de um sistema antes de sua implementação, permitindo uma análise mais aprofundada.



Tipos de Modelos em Modelagem de Sistemas

Modelos Estruturais:

Modelos estruturais de software mostram a organização de um sistema em termos de seus componentes que compõem este sistema e seus relacionamentos.

Podem ser estáticos, que mostram a estrutura do projeto do sistema, ou modelos dinâmicos, que mostram a organização do sistema quando ele está em execução.

Modelos Estruturais:

Você cria modelos estruturais de um sistema quando se está discutindo e projetando a arquitetura do sistema.

- ✓ Representam a arquitetura e a composição do sistema.
- ✓ Destacam componentes, entidades e suas relações.

Diagrama de Classes

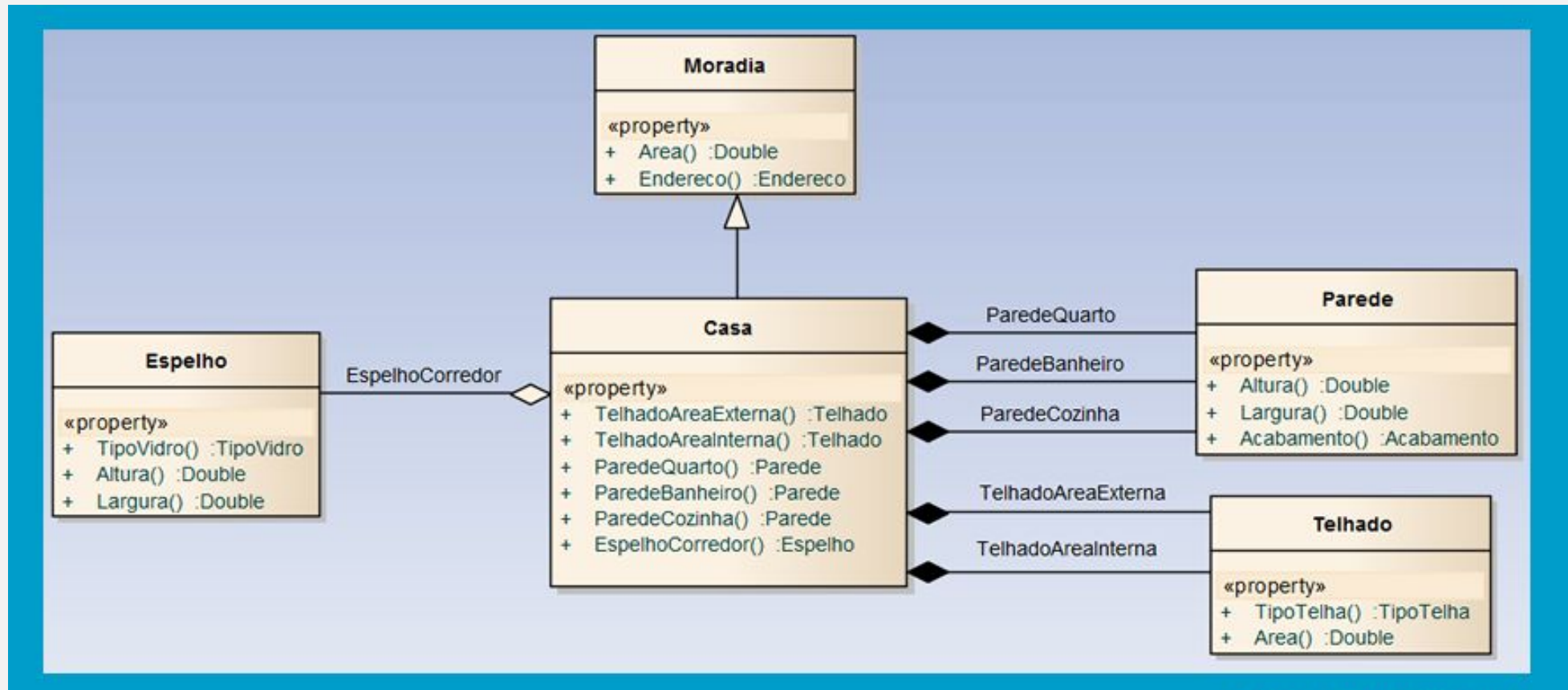


Diagrama de Classes

O Diagrama de Classes é uma ferramenta gráfica utilizada na modelagem de sistemas orientados a objetos.

Ao criar um Diagrama de Classes, se estabelece uma base sólida para o desenvolvimento de software orientado a objetos, garantindo uma representação visual clara e concisa da estrutura e das relações entre as diferentes partes do sistema.

Principais Componentes de um Diagrama de Classes:

Classe:

Representa uma categoria de objetos ou entidades no sistema. Por exemplo, em um sistema de biblioteca, pode haver uma classe chamada "Livro".

Atributos:

São características ou propriedades das classes. Cada classe possui atributos que descrevem suas características. Exemplos de atributos para a classe "Livro" podem incluir "Título", "Autor" e "Ano de Publicação".

Principais Componentes de um Diagrama de Classes:

Métodos:

São operações ou ações que uma classe pode realizar. Métodos definem o comportamento das classes. Para a classe "Livro", um método pode ser "RegistrarDevolução()".

Relacionamentos:

Descrevem as associações entre diferentes classes. Os relacionamentos indicam como as classes interagem umas com as outras. Exemplos de relacionamentos incluem associações, generalizações (herança) e composições.

Principais Componentes de um Diagrama de Classes:

Visibilidade:

Indica a visibilidade dos membros de uma classe (atributos e métodos) em relação a outras classes. Pode ser público (+), privado (-) ou protegido (#).

Multiplicidade:

Indica o número de instâncias de uma classe que podem se relacionar com instâncias de outra classe. Por exemplo, uma classe "Livro" pode estar associada a várias instâncias da classe "Empréstimo".

OBJETIVOS DE UM DIAGRAMA DE CLASSES

- ✓ Visualização da Estrutura Estática
- ✓ Modelagem Conceitual
- ✓ Projeto Orientado a Objetos
- ✓ Compreensão do Sistema
- ✓ Documentação

Diagrama de Relacionamento

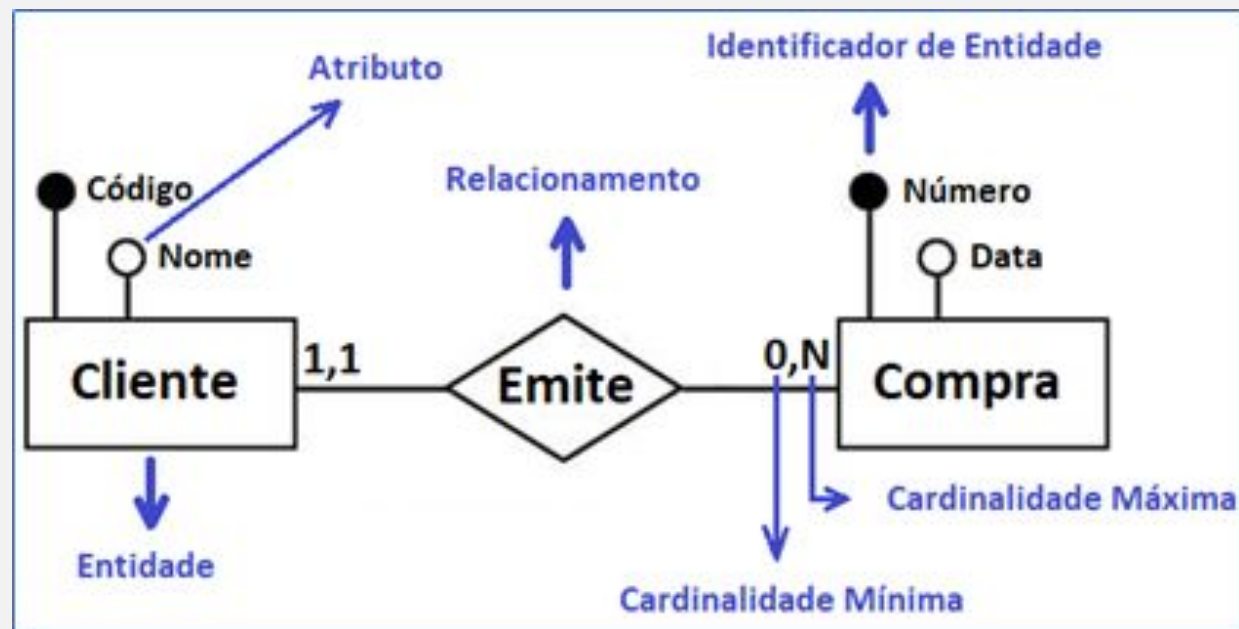


Diagrama de Entidade e Relacionamento (DER):

O Diagrama de Entidade e Relacionamento (DER) é uma representação gráfica utilizada na modelagem de dados para descrever as relações entre diferentes entidades em um sistema ou domínio específico. Ele é amplamente utilizado na área de Banco de Dados para projetar e visualizar a estrutura de dados de um sistema, facilitando a compreensão dos relacionamentos e interações entre diferentes partes do sistema.

Principais Componentes de um DER:

Entidades:

Representam objetos ou conceitos no mundo real que precisam ser armazenados no banco de dados. Por exemplo, em um sistema de biblioteca, as entidades podem incluir "Livro", "Usuário" e "Empréstimo".

Atributos:

São as características ou propriedades específicas de uma entidade. Cada entidade possui atributos que descrevem suas características. Por exemplo, um livro pode ter atributos como "Título", "Autor" e "Ano de Publicação".

Principais Componentes de um DER:

Relacionamentos:

Descrevem as conexões ou associações entre diferentes entidades. Os relacionamentos indicam como as entidades interagem umas com as outras. Por exemplo, o relacionamento "Livro - Empréstimo" indica que um livro pode ser emprestado e que um empréstimo envolve um ou mais livros.

Chaves Primárias:

São atributos ou conjuntos de atributos que identificam exclusivamente uma instância de uma entidade. Cada entidade geralmente tem uma chave primária que a distingue das outras instâncias. Por exemplo, o ISBN de um livro pode ser a chave primária da entidade "Livro".

OBJETIVOS DE UM DER

- ✓ Visualização de Relacionamentos
- ✓ Modelagem Conceitual
- ✓ Identificação de Requisitos
- ✓ Comunicação Efetiva

Tipos de Modelos em Modelagem de Sistemas

Modelos comportamentais:

Modelos comportamentais são modelos do comportamento dinâmico do sistema enquanto esse está em execução. Eles mostram o que acontece ou o que deveria acontecer quando um sistema responde a um estímulo de seu ambiente.

Modelos comportamentais:

Pode-se considerar esses estímulos como sendo de dois tipos:

Dados: Alguns dados que chegam e precisam ser processados pelo sistema.

Eventos: Alguns eventos que acontecem disparam o processamento do sistema. Os eventos podem ter dados associados, embora isso não ocorra sempre.

- ✓ Mostram como o sistema funciona e responde ao longo do tempo.
- ✓ Descrevem interações entre componentes e eventos.

Diagrama de Atividades

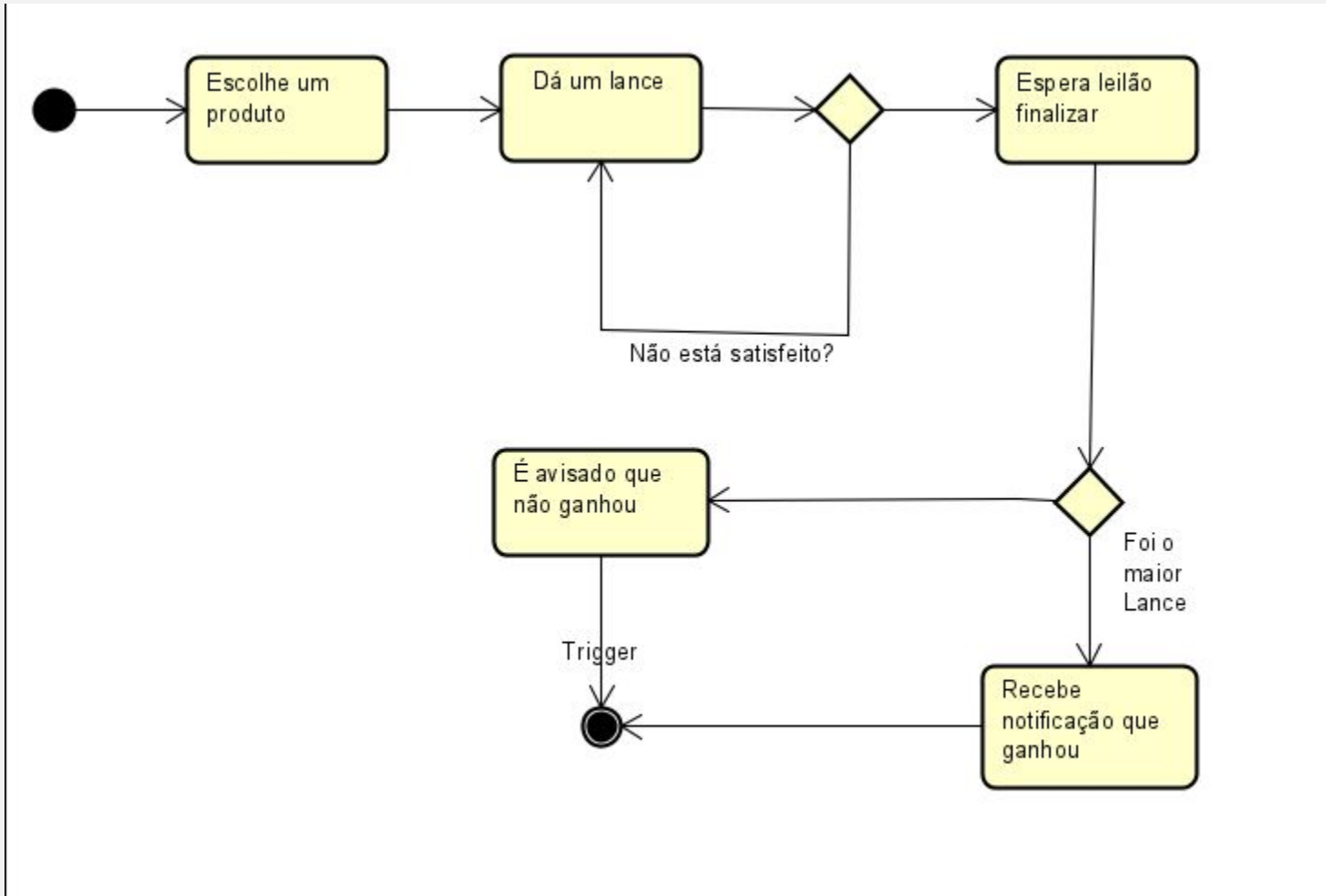


Diagrama de Atividades

O Diagrama de Atividades é uma ferramenta visual da Linguagem de Modelagem Unificada (UML) que representa o fluxo de controle de atividades em um sistema. Ele é utilizado para modelar processos de negócios, fluxos de trabalho, algoritmos e outros cenários nos quais a ênfase está nas atividades realizadas em um sistema.

Principais Componentes

Atividades:

Representam as unidades de trabalho executadas no sistema. Podem ser operações simples ou subprocessos mais complexos.

Nós de Decisão (Decision Nodes):

Indicam pontos de decisão no fluxo, onde o caminho a ser seguido depende de uma condição. São representados por losangos.

Bifurcações e Junções (Fork and Join Nodes):

Bifurcações representam a divisão do fluxo em múltiplos caminhos, enquanto junções indicam a reunião desses caminhos.

Principais Componentes

Fluxo de Controle (Control Flow):

Linhas direcionadas que conectam atividades e outros elementos, mostrando a sequência de execução.

Nós de Inicialização e Término (Initial and Final Nodes):

Indicam o início e o fim do fluxo de atividades. O nó de inicialização é geralmente representado por um círculo, enquanto o nó de término é um ponto.

Principais Componentes

Partições (Swimlanes):

Permitem organizar as atividades em grupos lógicos, chamados de partições ou swimlanes, que representam diferentes participantes, sistemas ou áreas funcionais.

Transições:

São setas ou linhas conectando diferentes elementos do diagrama, representando a passagem de controle entre atividades.

Objetivos

- ✓ Modelagem de Processos
- ✓ Identificação de Sequências
- ✓ Análise de Decisões
- ✓ Visualização de Fluxos Paralelos
- ✓ Compreensão de Fluxos de Controle
- ✓ Documentação e Comunicação

Diagrama de Caso de Uso

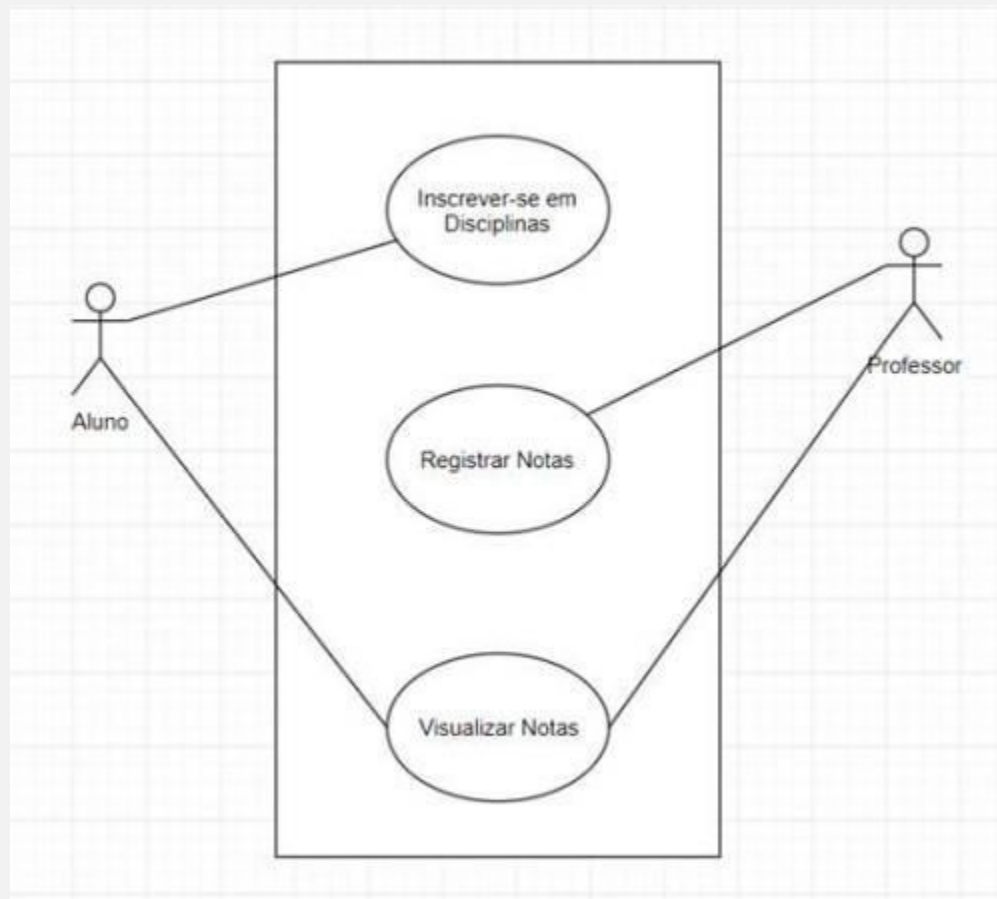


Diagrama de Casos de Uso

O Diagrama de Casos de Uso é outra representação gráfica da Linguagem de Modelagem Unificada (UML) que destaca a interação entre um sistema e seus atores externos. Ele é amplamente utilizado para modelar os requisitos funcionais de um sistema e identificar as diferentes maneiras pelas quais os usuários interagem com o sistema.

Fornece uma visão centrada no usuário das funcionalidades que o sistema deve oferecer. Ele é utilizado como parte integrante da documentação de requisitos e desempenha um papel essencial na comunicação entre a equipe de desenvolvimento e os stakeholders.

Principais Componentes

Ator:

Representa um papel ou entidade externa que interage com o sistema. Pode ser um usuário humano, outro sistema, ou até mesmo um dispositivo externo.

Caso de Uso:

Representa uma funcionalidade ou serviço específico que o sistema oferece aos seus usuários (atores). Cada caso de uso descreve uma interação específica entre um ator e o sistema.

Principais Componentes

Relacionamentos de Associação:

Linhas que conectam atores aos casos de uso, indicando a relação entre eles. Uma associação mostra que um ator está envolvido em um ou mais casos de uso.

Relacionamentos de Inclusão e Extensão:

Representam relações entre diferentes casos de uso. A inclusão indica que um caso de uso inclui outro, enquanto a extensão indica uma extensão ou variação de um caso de uso básico.

Principais Componentes

Relacionamento de Generalização:

Indica herança entre casos de uso, mostrando que um caso de uso mais específico herda comportamentos de um caso de uso mais genérico.

Sistema:

Representa o sistema em si, mostrando os limites do sistema em relação aos atores externos.

Objetivos

- ✓ Identificação de Requisitos Funcionais
- ✓ Visualização da Interação
- ✓ Compreensão do Escopo
- ✓ Priorização de Desenvolvimento
- ✓ Comunicação com Stakeholders
- ✓ Base para Testes

PARTE II

UMA BREVE HISTÓRIA DA UML

Resumo da Origem e Evolução da UML

A UML (Unified Modeling Language) tem suas raízes na compilação das "melhores práticas de engenharia" provenientes de métodos como Booch, OMT (Rumbaugh) e OOSE (Jacobson). Estes métodos foram unificados para criar uma linguagem comum de modelagem para sistemas grandes e complexos.

Objetivo da UML:

Tornar-se a linguagem padrão para modelar sistemas concorrentes e distribuídos.

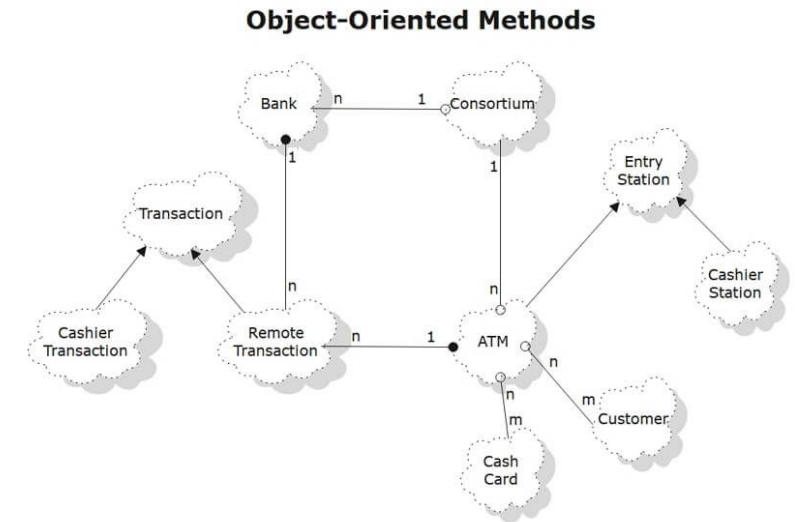
Modelo de Booch

Um objeto = um modelo do mundo real com dados e habilidades;

✓ Desenho estruturado e OO;

Estruturado: Separa o sistema em módulos;

OO: Problema com os objetos que existem em um dominio de problema;



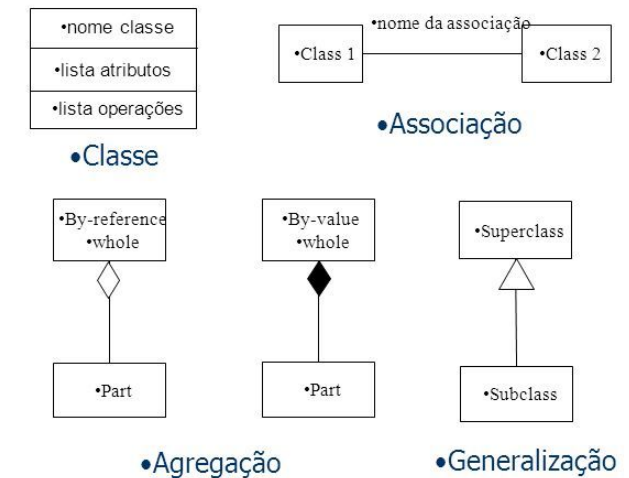
Modelo OMT

Modelo OMT - Object Modeling Technique

- ✓ método de projeto orientado a objetos
- ✓ modelo de objetos captura a estrutura estática do sistema: classes, relacionamentos, e operações.

Conceitos:

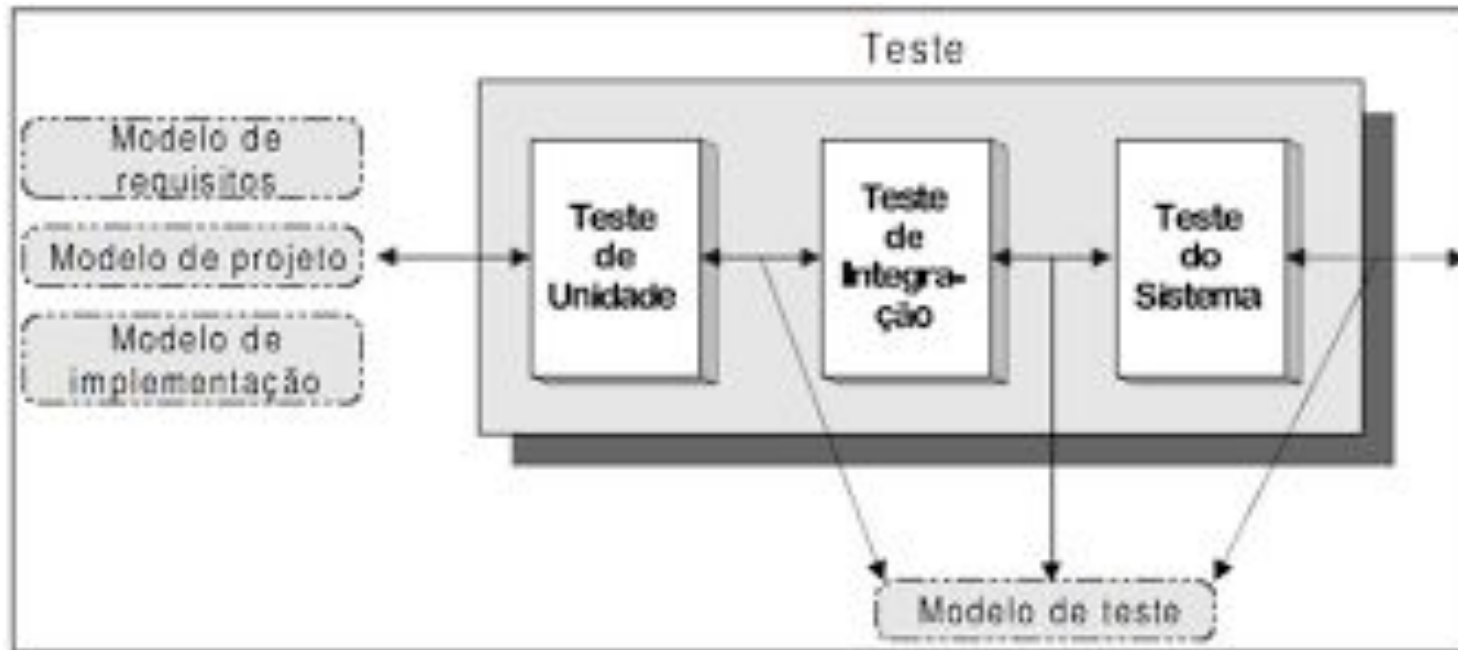
- ✓ objeto - entidade do mundo real
- ✓ classe de objetos - representa entidades de mesma característica (atributos, operações)
- ✓ associações - relacionamento entre objetos
- ✓ generalização - hierarquia entre classes
- ✓ agregação - combinação de outras partes



Método OOSE:

Identificar os usuários e suas responsabilidades

- ✓ Construir modelo de requisitos, atores e responsabilidades casos de uso para cada ator
- ✓ Construir modelo de análise



EVOLUÇÃO

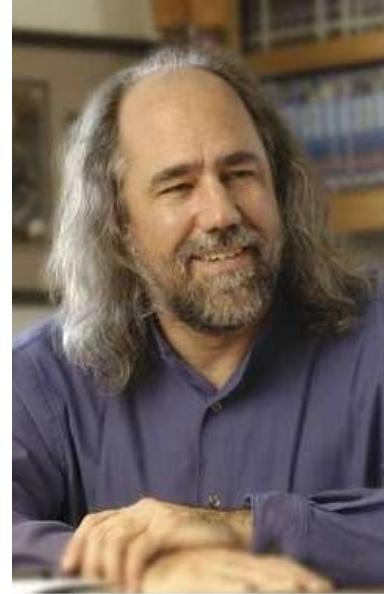
- ✓ **Outubro de 1994:** Booch e Rumbaugh se uniram na Rational.
- ✓ **Outubro de 1995:** lançamento do esboço da versão 0.8 do Unified Process, visando unificar o método de Booch e OMT.
- ✓ **Junho de 1996:** a versão 0.9 da UML foi lançada, incorporando também o método OOSE de Jacobson.
- ✓ **1997:** a UML foi aprovada como padrão pelo Object Management Group (OMG), um consórcio internacional de empresas que define padrões na área de Orientação a Objetos.

Rational® software

CONTRIBUENTES

Grandes contribuições dos pesquisadores Grandy Booch, James Rumbaugh e Ivar Jacobson, conhecidos como "os três amigos".

A construção da UML buscou aproveitar características das notações preexistentes, criando uma linguagem mais expressiva.



grady booch



ivar jacobson



james rumbaugh

PARTE III

Importância da Modelagem na Engenharia de Software

modelagem desempenha um papel central ao fornecer uma representação visual e estruturada que apoia a comunicação, compreensão, detecção precoce de problemas e tomada de decisões informada ao longo do ciclo de vida do software. Sua aplicação inteligente contribui para o sucesso de projetos de engenharia de software.

A sua importância é evidente em várias dimensões:

Compreensão do Sistema:

- ✓ Através da modelagem, é possível compreender a complexidade de um sistema antes de sua implementação.
- ✓ Facilita a visualização e análise de componentes, estrutura e interações.

Comunicação Efetiva:

- ✓ Proporciona uma linguagem comum entre stakeholders, incluindo desenvolvedores, designers e clientes.
- ✓ Ajuda na comunicação efetiva de ideias e requisitos, reduzindo mal-entendidos.

Detecção Antecipada de Problemas:

- ✓ Possibilita identificar potenciais problemas e inconsistências no design antes da implementação.
- ✓ Reduz riscos ao permitir ajustes antes que custos significativos sejam incorridos.

Tomada de Decisões Embasada:

- ✓ Modelos fornecem uma base sólida para a tomada de decisões, orientando escolhas de arquitetura, design e implementação.
- ✓ Facilita a avaliação de alternativas e a escolha da abordagem mais adequada.

Documentação Estruturada:

- ✓ Serve como documentação visual que pode ser referenciada ao longo do ciclo de vida do software.
- ✓ Facilita a compreensão para desenvolvedores futuros, mantendo um registro claro do design inicial.

Aprimoramento Contínuo:

✓ Modelos podem ser iterados e aprimorados à medida que o entendimento do sistema evolui.

Suporta o processo de desenvolvimento ágil, permitindo ajustes conforme novos requisitos surgem.

Economia de Recursos:

✓ Reduz custos ao evitar retrabalho e correções tardias.

✓ Contribui para a eficiência geral do processo de desenvolvimento.

PARTE IV

PRATICAR

HANDs ON



BORA PARA UMA DINÂMICA

Estudo de Caso

Sistema de Controle de Biblioteca Universitária

Cenário:

A Universidade XYZ deseja melhorar a eficiência da sua biblioteca, que atualmente enfrenta desafios na organização e gestão de seu acervo. O objetivo é implementar um sistema que simplifique o processo de controle de livros, usuários, empréstimos e devoluções.

Estudo de Caso

Requisitos Iniciais:

- Cadastro de Livros
- Cadastro de Usuários
- Empréstimos e Devoluções
- Controle de Prazos
- Relatórios

Atividades:

- Identificação de Entidades e Relacionamentos
- Definição de Requisitos Adicionais

Muito obrigada!



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC